Sri Sivasubramaniya Nadar College of Engineering

Department of Computer Science and Engineering

UCS3402 — Database Management Systems

Assignment I and II — Report

INSURANCE CLAIM PROCESSING SYSTEM (ICPS)

Document-Based NoSQL Database (MongoDB)

Submitted by:

Sankar Shailesh	3122245001138
S. Sree Krishna	3122245001162
M. Sujith	3122245001172

Submitted to:

Dr. P. Mirunalini

Department of Computer Science and Engineering

Class: CSE **Section:** C

Academic Year: 2025–2026 — Semester: IV

Contents

1 Introduction **3**

2 Problem Statement **3**

3 System Scope and Constraints **3**

3.1 Scope . . . 3

3.2 Constraints . . . 4

4 Identification of Entities, Attributes, and Relationships **4**

4.1 Design Procedure Adopted . . . 4

4.2 Justification of Identified Entities . . . 6

4.3 Justification of Relationships . . . 8

5 Conceptual ER Diagram **9**

6 Entities with Attributes, Data Types, and Keys **17**

6.1 Identified Collections (Entities) . . . 17

6.2 Entity 1: PolicyHolder . . . 17

6.3 Entity 2: Branch . . . 18

6.4 Entity 3: Event (Policy Type Classifier) . . . 19

6.5 Entity 4: Policy . . . 20

6.6 Entity 5: ServiceProvider . . . 21

6.7 Entity 6: Claim (with Embedded Documents and Status History) . . . 21

6.8 Entity 7: Surveyor . . . 23

6.9 Entity 8: ClaimReview . . . 24

6.10 Entity 9: Payment . . . 25

6.11 Summary of Entities and Storage . . . 25

6.12 Key Constraints Summary . . . 26

7 Relationships with Types, Implementation, and Participation **27**

7.1 PolicyHolder – Policy (HAS) . . . 27

7.2 Branch – Policy (MANAGES) . . . 27

7.3 Event – Policy (CATEGORIZES) . . . 27

7.4 Policy – Claim (FILES) . . . 27

7.5 Claim – ServiceProvider (HANDLES) . . . 28

7.6 Event – Claim (TRIGGERS) . . . 28

7.7 Claim – ClaimDocument (HAS_DOCUMENT) . . . 28

7.8 Claim – ClaimStatusHistory (HAS_HISTORY) . . . 28

7.9 Claim – ClaimReview (REVIEWED_BY) 28
7.10 Surveyor – ClaimReview (PERFORMS) 28
7.11 Claim – Payment (SETTLED_BY) 28
7.12 Final Relationship Summary Table 28

8 Assignment 1 **29**
8.1 (a) Identify Entities, Attributes, and Relationships 29
8.2 (b) Conceptual Diagram (NoSQL High-Level Model) 32
8.3 (c) Map to Physical Schema 33
8.4 (d) Functional Dependencies, Query Patterns, and Key Identification . . . 36

9 Functional Dependencies and Key Identification **39**
9.1 Introduction 39
9.2 Functional Dependencies per Collection 39
9.3 Derivation of Minimal Cover 41
9.4 Key Identification 42

10 Assignment 2: Optimised NoSQL Database Schema Design **42**
10.1 Objective of Schema Optimisation 42
10.2 Frequently Executed Query Patterns 42
10.3 Optimised Claims Collection Schema 43
10.4 Embedding vs Referencing Optimisation Decisions 43
10.5 Denormalisation Strategy 44
10.6 Index Optimisation 44
10.7 Partition Key and Scalability 45
10.8 Performance Optimisation Benefits 45
10.9 Data Consistency Considerations 45
10.10 Final Optimised Schema Architecture Summary 45

11 Implementation Details **46**
11.1 Collection Initialisation 46
11.2 Sample CRUD Operations 46

12 Conclusion **49**

13 Team Member Contributions **49**

1. Introduction

The Insurance Claim Processing System (ICPS) is a large-scale enterprise system used by insurance companies to manage policyholders, insurance policies, claim submissions, verification processes, and final approval or rejection of claims. The system ensures data integrity, consistency, and efficient claim lifecycle management across multiple insurance product lines including Medical, Vehicle, Property/Fire, Life Insurance, Natural Disaster, and Theft policies.

This report documents the complete database design for ICPS across two assignments. Assignment 1 covers the conceptual design using an Entity-Relationship (ER) model, identification and justification of entities and relationships, and the ER diagram. Assignment 2 extends the design to a document-based NoSQL database (MongoDB), covering NoSQL schema design, physical collection schemas, optimisation strategies, and CRUD operations.

2. Problem Statement

The objective of this project is to design a comprehensive database schema for the Insurance Claim Processing System (ICPS). The system must support policyholder registration, policy management, claim submission, verification and approval workflows, and reporting, while maintaining data integrity and minimising redundancy.

For Assignment 1, the design uses a traditional relational database modelling approach with an ER diagram. For Assignment 2, the design is extended to a document-based NoSQL database (MongoDB) to leverage schema flexibility, embedded document support, and horizontal scalability.

The system must handle:

- Multiple insurance product types through a flexible Event-based classification mechanism
- Branch-level policy management
- Third-party service provider tracking for claims
- Complete claim status audit trail
- Claim verification by authorised surveyors
- Payment settlement for approved claims

3. System Scope and Constraints

3.1 Scope

The system includes:

- Policyholder registration and management

- Branch management and association with policies
- Event-based flexible insurance policy categorisation (Medical, Vehicle, Property/-Fire, Life, Natural Disaster, Theft)
- Insurance policy management
- Service provider tracking (hospitals, garages, inspection firms)
- Claim submission and tracking
- Claim document management
- Claim review and approval process
- Surveyor and claim verification handling
- Claim status history and audit trail
- Payment settlement for approved claims

3.2 Constraints

- Each claim must be associated with exactly one policy.
- Each policy must be linked to exactly one Event (policy type classifier).
- Each policy must be issued by exactly one Branch.
- A policyholder may own multiple policies.
- A policy can have multiple claims.
- A claim must be reviewed before approval or rejection.
- Each claim must be associated with exactly one Service Provider.
- Each claim is triggered by exactly one Event type.
- Claim status history must be maintained for auditing purposes.
- Unique identifiers must be maintained for all major entities.
- Aadhaar number, policy number, claim number, licence number, and transaction reference number must be globally unique.

4. Identification of Entities, Attributes, and Relationships

4.1 Design Procedure Adopted

The database design for the ICPS was developed using a structured conceptual modelling approach to ensure completeness, correctness, and real-world applicability.

Requirement Analysis

The problem statement was carefully analysed to extract key business objects and operational activities. Important nouns such as policyholders, policies, branches, events, claims, service providers, surveyors, reviews, status histories, and payments were identified as potential entities. The interactions between these objects were studied to understand

workflow dependencies across the claim lifecycle.

Identification of Core Business Entities

Entities were selected based on the following criteria:

- The object must have independent existence.
- The object must store persistent data.
- The object must have a unique identifier.
- The object must participate in system transactions.

Identification of Attributes

Attributes were determined based on:

- Information required to uniquely identify each entity.
- Data necessary for verification, validation, and claim processing.
- Attributes required for auditing and reporting.
- Elimination of redundant or derived storage wherever possible.

Identification of Relationships

Relationships were derived from business rules such as:

- A policyholder owns one or more policies.
- A branch manages one or more policies.
- An event categorises one or more policies.
- A policy can have multiple claims.
- A claim is handled by a service provider.
- An event triggers a claim.
- A claim has supporting documents.
- A claim has a status history.
- A surveyor reviews claims through a ClaimReview.
- Approved claims are settled through payment.

Classification of Attribute Types

Attributes were classified as:

- **Simple attributes** — atomic, indivisible values
- **Composite attributes** — Address (consisting of street, city, state, pincode)
- **Multivalued attributes** — Phone (a policyholder may have multiple contact numbers)
- **Derived attributes** — Age (calculated from DOB), PolicyDuration (from StartDate and EndDate), ClaimProcessingTime (from submission to decision)

- **Primary and candidate keys** — unique identifiers for each entity

Constraint Identification

Participation constraints and cardinality ratios were assigned according to real-world insurance operations to ensure data consistency and integrity. All entities in this design are strong entities with their own primary keys. This systematic approach ensures that the ER model accurately represents the real-world Insurance Claim Processing workflow.

4.2 Justification of Identified Entities

PolicyHolder

The PolicyHolder entity represents customers who purchase insurance policies and submit claims. This entity is fundamental because the claim lifecycle originates from the policyholder.

PolicyHolderID uniquely identifies each customer. AadhaarNo is a candidate key for verification. **Phone** is a multivalued attribute. **Address** is a composite attribute. **Age** is derived from DOB.

PolicyHolder is modelled as a **strong entity** with its own primary key.

Branch

The Branch entity represents regional offices of the insurance company responsible for issuing and managing policies. Branch supports regional reporting and managerial accountability.

BranchID uniquely identifies each branch. Branch is modelled as a **strong entity** in a 1:N relationship with Policy.

Event

The Event entity acts as a universal policy type classification mechanism, eliminating the need for separate subclass tables. New insurance products are introduced by adding a new Event record only.

EventID uniquely identifies each event type. **RiskLevel** supports underwriting decisions. Event is a **strong entity** participating in relationships with both Policy (CATEGORIZES) and Claim (TRIGGERS).

Policy

The Policy entity represents insurance agreements issued to policyholders through a specific branch, categorised by an event type. Claims are always linked to a valid active policy.

PolicyID is the primary key. **PolicyNumber** is a candidate key. **PolicyDuration** is derived from StartDate and EndDate. Policy is a **strong entity**.

ServiceProvider

The ServiceProvider entity represents third-party organisations (hospitals, vehicle repair centres, property inspection firms) whose services are associated with claims.

ProviderID is the primary key. **LicenseNo** is a candidate key ensuring regulatory compliance. ServiceProvider is a **strong entity** in a 1:N relationship with Claim.

Claim

The Claim entity is the central transactional component of the ICPS. Claims must undergo validation, verification, and status tracking.

ClaimID is the primary key. **ClaimNumber** is a candidate key. **ClaimStatus** represents workflow stages. **ClaimProcessingTime** is derived. Claim carries **EventID** and **ProviderID** as foreign key attributes.

Claim is a **strong entity** connected to Policy, ClaimDocument, ClaimStatusHistory, ClaimReview, ServiceProvider, Event, and Payment.

ClaimDocument

The ClaimDocument entity represents supporting documents for claim verification. Each document belongs to exactly one claim and is identified by its own **DocumentID** as a primary key.

ClaimDocument is modelled as a **strong entity** (updated from the previous weak entity design to reflect that each document has its own unique identifier independent of the composite key).

ClaimStatusHistory

The ClaimStatusHistory entity represents the audit trail of status transitions for each claim. Each status entry is uniquely identified by its own **StatusID**.

ClaimStatusHistory is modelled as a **strong entity** (updated from the previous weak entity design), critical for regulatory compliance and dispute resolution.

Surveyor

The Surveyor entity represents authorised personnel responsible for claim verification and assessment. **SurveyorID** is the primary key. LicenseNo is a candidate key.

Surveyor is a **strong entity** participating in the claim review process.

ClaimReview

ClaimReview represents the review activity performed by surveyors on claims, resolving the many-to-many relationship between Claim and Surveyor.

ReviewID is the primary key. ClaimReview stores ReviewDate, RecommendedAmount, and Remarks.

ClaimReview is an **associative strong entity**.

Payment

The Payment entity represents financial settlement for approved claims. Payment is optional (only for approved claims).

PaymentID is the primary key. TransactionRefNo is a candidate key. Payment is a **strong entity** in a 1:1 optional relationship with Claim.

4.3 Justification of Relationships

- A **PolicyHolder** *owns* one or more **Policies** (1:N). PolicyHolder — Partial; Policy — Total.
- A **Branch** *manages* one or more **Policies** (1:N). Branch — Partial; Policy — Total.
- An **Event** *categorises* one or more **Policies** (1:N). Event — Partial; Policy — Total.
- A **Policy** *generates* multiple **Claims** (1:N). Policy — Partial; Claim — Total.
- A **Claim** is *handled by* one **ServiceProvider** (N:1). Claim — Total; ServiceProvider — Partial.
- An **Event** *triggers* one or more **Claims** (1:N). Event — Partial; Claim — Total.
- A **Claim** *has* multiple **ClaimDocuments** (1:N). Both participate; ClaimDocument — Total.
- A **Claim** *has* multiple **ClaimStatusHistory** entries (1:N). Both — Total.
- A **Claim** is *reviewed by* Surveyors through **ClaimReview** (M:N resolved). Claim — Partial; ClaimReview — Total.
- A **Surveyor** *performs* reviews through **ClaimReview** (1:N). Surveyor — Partial; ClaimReview — Total.
- A **Claim** may be *settled by* one **Payment** (1:1). Claim — Partial; Payment — Total.

5. Conceptual ER Diagram

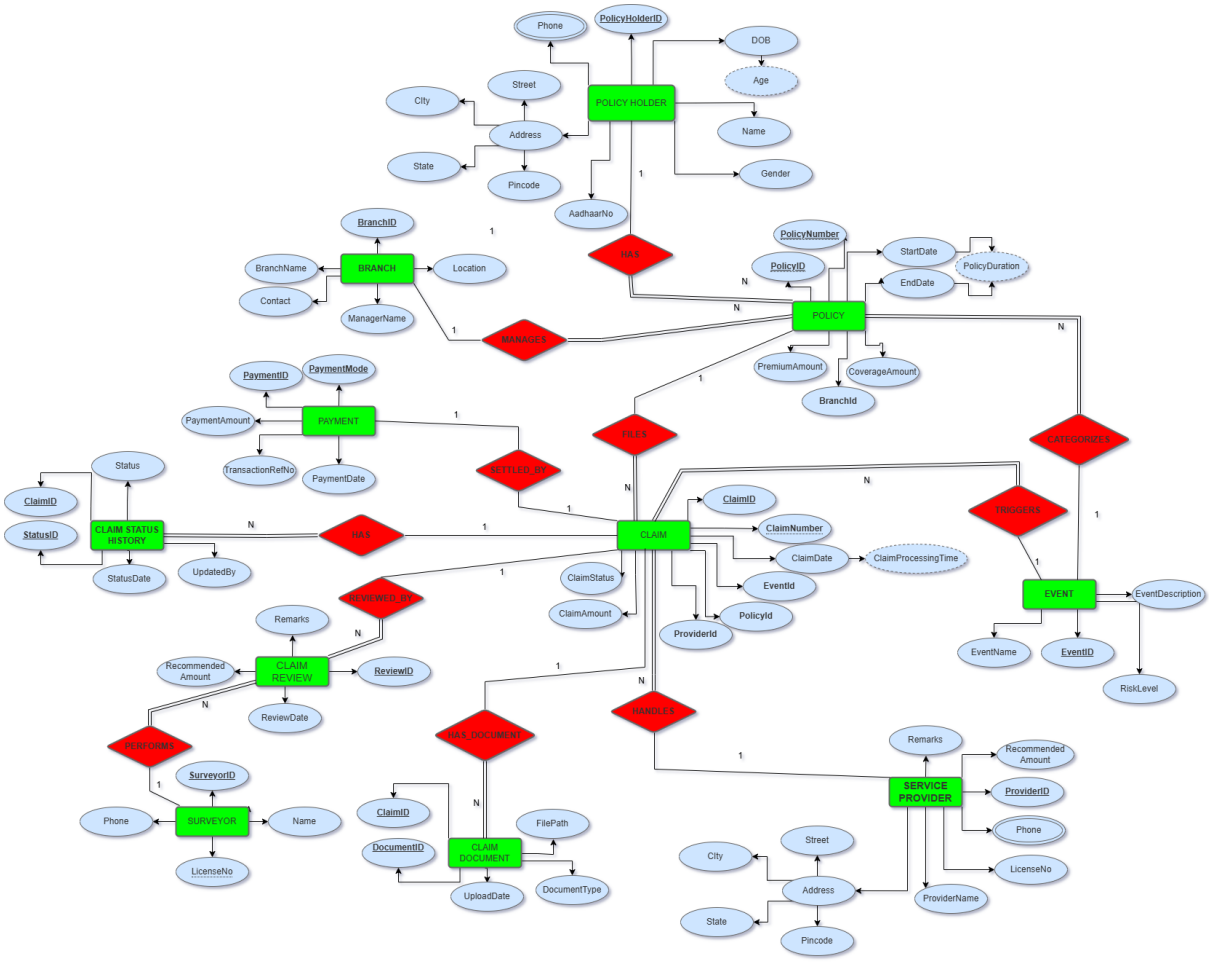

Figure 1. *ER Diagram for ICPS*

Entity 1: PolicyHolder

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
PolicyHolderID	Unique identifier	Simple	Primary Key
Name	Full name	Simple	—
DOB	Date of birth	Simple	—
Age	Calculated from DOB	Derived	—
Gender	Gender	Simple	—
AadhaarNo	National ID number	Simple	Candidate Key
Phone	Contact numbers	Multivalued	—
Address	Residential address	Composite	—

Attribute	Description	Attribute Type	Key Type
Street	Street name	Simple (Part of Address)	—
City	City name	Simple (Part of Address)	—
State	State name	Simple (Part of Address)	—
Pincode	Postal code	Simple (Part of Address)	—

Entity 2: Branch

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
BranchID	Unique branch identifier	Simple	Primary Key
BranchName	Name of branch	Simple	—
Location	City / location	Simple	—
ManagerName	Branch manager	Simple	—
Contact	Branch contact number	Simple	—

Entity 3: Event

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
EventID	Unique event identifier	Simple	Primary Key
EventName	Insurance type name	Simple	—
EventDescription	Description of coverage	Simple	—
RiskLevel	Risk classification	Simple	—

Entity 4: Policy

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
PolicyID	Unique policy identifier	Simple	Primary Key
PolicyNumber	Unique policy number	Simple	Candidate Key
PremiumAmount	Premium payable	Simple	—
CoverageAmount	Coverage amount	Simple	—
StartDate	Policy start date	Simple	—
EndDate	Policy end date	Simple	—
PolicyDuration	Derived from Start/End Date	Derived	—
BranchID	Issuing branch (FK)	Simple	Foreign Key

Entity 5: ServiceProvider

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
ProviderID	Unique provider identifier	Simple	Primary Key
ProviderName	Name of provider	Simple	—
ProviderType	Hospital / Garage / Inspection	Simple	—
Address	Provider address	Composite	—
Phone	Contact number	Simple	—
LicenseNo	Professional licence number	Simple	Candidate Key
RecommendedAmount	Recommended settlement	Simple	—
Remarks	Notes on provider	Simple	—

Entity 6: Claim

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
ClaimID	Unique claim identifier	Simple	Primary Key
ClaimNumber	Unique claim reference	Simple	Candidate Key
ClaimDate	Date claim was filed	Simple	—
ClaimAmount	Claimed amount	Simple	—
ClaimStatus	Current workflow status	Simple	—
ClaimProcessingTime	Time taken to process	Derived	—
EventID	Event type (FK)	Simple	Foreign Key
PolicyID	Related policy (FK)	Simple	Foreign Key
ProviderID	Service provider (FK)	Simple	Foreign Key

Entity 7: ClaimDocument

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
DocumentID	Unique document identifier	Simple	Primary Key
ClaimID	Parent claim (FK)	Simple	Foreign Key
FilePath	File storage path	Simple	—
DocumentType	Type of document	Simple	—
UploadDate	Date uploaded	Simple	—

Entity 8: ClaimStatusHistory

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
StatusID	Unique status entry identifier	Simple	Primary Key
ClaimID	Parent claim (FK)	Simple	Foreign Key
Status	Status value	Simple	—
StatusDate	Date of status change	Simple	—

Attribute	Description	Attribute Type	Key Type
UpdatedBy	Who made the update	Simple	—

Entity 9: ClaimReview

Entity Type: Associative Strong Entity

Attribute	Description	Attribute Type	Key Type
ReviewID	Unique review identifier	Simple	Primary Key
ClaimID	Reviewed claim (FK)	Simple	Foreign Key
SurveyorID	Reviewing surveyor (FK)	Simple	Foreign Key
ReviewDate	Date of review	Simple	—
RecommendedAmount	Recommended settlement	Simple	—
Remarks	Review comments	Simple	—

Entity 10: Surveyor

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
SurveyorID	Unique surveyor identifier	Simple	Primary Key
Name	Name of surveyor	Simple	—
LicenseNo	Licence number	Simple	Candidate Key
Phone	Contact number	Simple	—

Entity 11: Payment

Entity Type: Strong Entity

Attribute	Description	Attribute Type	Key Type
PaymentID	Unique payment identifier	Simple	Primary Key

Attribute	Description	Attribute Type	Key Type
ClaimID	Settled claim (FK)	Simple	Foreign Key
PaymentDate	Date of payment	Simple	—
PaymentAmount	Amount paid	Simple	—
PaymentMode	Mode of payment	Simple	—
TransactionRefNo	Bank transaction reference	Simple	Candidate Key

Attribute Types Used

Attribute Type	Used In
Simple	Most attributes across all entities
Composite	Address (PolicyHolder, ServiceProvider)
Multivalued	Phone (PolicyHolder)
Derived	Age, PolicyDuration, ClaimProcessingTime
Primary Key	All strong entities
Candidate Key	AadhaarNo, PolicyNumber, ClaimNumber, LicenseNo (Surveyor & ServiceProvider), TransactionRefNo
Foreign Key	BranchID (Policy), EventID and PolicyID and ProviderID (Claim), ClaimID (ClaimDocument, ClaimStatusHistory, ClaimReview, Payment), SurveyorID (ClaimReview)

Relationship Documentation

No	Relationship	Entity 1	Entity 2	Type	Description
1	HAS	PolicyHolder	Policy	Binary	Policyholder owns policies
2	MANAGES	Branch	Policy	Binary	Branch issues policies
3	CATEGORIZES	Event	Policy	Binary	Event classifies policy type
4	FILES	Policy	Claim	Binary	Policy generates claims
5	HANDLES	Claim	ServiceProvider	Binary	Claim linked to service provider

No	Relationship	Entity 1	Entity 2	Type	Description
6	TRIGGERS	Event	Claim	Binary	Event type triggers a claim
7	HAS_DOCUMENT	Claim	ClaimDocument	Binary	Claim has supporting documents
8	HAS_HISTORY	Claim	ClaimStatusHis	Binary	Claim has status audit trail
9	REVIEWED_BY	Claim	ClaimReview	Binary	Claim has reviews
10	PERFORMS	Surveyor	ClaimReview	Binary	Surveyor performs review
11	SETTLED_BY	Claim	Payment	Binary	Claim settled through payment

Participation and Cardinality Documentation

No	Relationship	Entity	Min	Max	Participati	Cardinality
1	HAS	PolicyHolder	0	N	Partial	1:N
		Policy	1	1	Total	
2	MANAGES	Branch	0	N	Partial	1:N
		Policy	1	1	Total	
3	CATEGORIZES	Event	0	N	Partial	1:N
		Policy	1	1	Total	
4	FILES	Policy	0	N	Partial	1:N
		Claim	1	1	Total	
5	HANDLES	Claim	1	N	Total	N:1
		ServiceProvider	0	N	Partial	
6	TRIGGERS	Event	0	N	Partial	1:N
		Claim	1	1	Total	
7	HAS_DOCUMENT	Claim	0	N	Partial	1:N
		ClaimDocument	1	1	Total	
8	HAS_HISTORY	Claim	1	N	Total	1:N
		ClaimStatusHis	1	1	Total	
9	REVIEWED_BY	Claim	0	N	Partial	1:N
		ClaimReview	1	1	Total	
10	PERFORMS	Surveyor	0	N	Partial	1:N
		ClaimReview	1	1	Total	
11	SETTLED_BY	Claim	0	1	Partial	1:1

No	Relationship	Entity	Min	Max	Participati	Cardinality
		Payment	1	1	Total	

6. Entities with Attributes, Data Types, and Keys

6.1 Identified Collections (Entities)

In a document-based NoSQL database, entities are represented as collections. The ICPS system contains nine primary collections and four embedded entities.

Main Collections

- `policyholders`
- `branches`
- `events`
- `policies`
- `service_providers`
- `claims`
- `surveyors`
- `claim_reviews`
- `payments`

Embedded Entities

- `phones[]` — embedded array inside `policyholders`
- `address{}` — embedded sub-document inside `policyholders` and `service_providers`
- `documents[]` — embedded array inside `claims` (reclassified from strong entity ClaimDocument)
- `status_history[]` — embedded array inside `claims` (reclassified from strong entity ClaimStatusHistory)

6.2 Entity 1: PolicyHolder

Description: Represents registered customers. Origin of the entire claim lifecycle.

Collection Name: `policyholders`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique policyholder identifier
`name`	String	—	Full name
`dob`	Date	—	Date of birth
`gender`	String	—	M / F / Other
`aadhaar_no`	String	Unique Key	National ID number
`phones`	Array<String>	—	Embedded contact numbers array

Attribute	Data Type	Key Type	Description
address.street	String	—	Street (sub-document)
address.city	String	—	City (sub-document)
address.state	String	—	State (sub-document)
address.pincode	String	—	Pincode (sub-document)

Listing 1. *PolicyHolder Document Example*

```
{
  "_id": "PH001",
  "name": "Arjun Kumar",
  "dob": ISODate("1990-05-12"),
  "gender": "M",
  "aadhaar_no": "456789123456",
  "phones": ["9876543210"],
  "address": {
    "street": "Anna Nagar",
    "city": "Chennai",
    "state": "Tamil Nadu",
    "pincode": "600040"
  }
}
```

Design Note: `phones[]` replaces the relational `PolicyHolderPhone` table. `address{}` replaces the BCNF-decomposed `Location` table. Both are stored natively within the document, eliminating joins.

6.3 Entity 2: Branch

Description: Regional offices of the insurance company. Enables regional reporting and accountability.

Collection Name: `branches`

Attribute	Data Type	Key Type	Description
_id	String	Primary Key	Unique branch identifier
branch_name	String	—	Full name of branch
location	String	—	City / location
manager_name	String	—	Branch manager name
contact	String	—	Branch contact number

Listing 2. *Branch Document Example*

```
{
  "_id": "B101",
```

```
  "branch_name": "Chennai Central Branch",
  "location": "Chennai",
  "manager_name": "R. Kumar",
  "contact": "04424567890"
}
```

6.4 Entity 3: Event (Policy Type Classifier)

Description: Universal policy type classification mechanism. Also triggers claims. Eliminates subclass tables.

Collection Name: `events`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique event identifier
`event_name`	String	—	Insurance event/type name
`event_description`	String	—	Description of coverage scope
`risk_level`	String	—	Medium / High / Very High

Listing 3. *Event Document Example*

```
{
  "_id": "E101",
  "event_name": "Medical Treatment",
  "event_description": "Hospitalization, surgeries, medical bills",
  "risk_level": "Medium"
}
```

All six insurance product types managed through this single collection:

EventI	Event Name	Policy Type	Description	Risk
E101	Medical Treatment	Medical Policy	Hospitalisation, surgery	Medium
E102	Vehicle Accident	Vehicle Policy	Road accidents, damage	High
E103	Fire Damage	Property Policy	Fire to property	High
E104	Natural Disaster	Disaster Policy	Flood, earthquake	Very High
E105	Life Insurance	Life Policy	Death benefit	High

EventI	Event Name	Policy Type	Description	Risk
E106	Theft	Theft Policy	Theft of insured property	Medium

Design Note: Event participates in two relationships — CATEGORIZES with Policy and TRIGGERS with Claim. Both Policy and Claim carry `event_id` as a reference key, allowing the insurance product type to be determined from either direction.

6.5 Entity 4: Policy

Description: Insurance agreements issued to policyholders through a branch, categorised by an event type.

Collection Name: `policies`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique policy identifier
`policy_number`	String	Unique Key	Human-readable policy number
`premium_amount`	Decimal	—	Monthly/annual premium
`coverage_amount`	Decimal	—	Maximum coverage amount
`start_date`	Date	—	Policy start date
`end_date`	Date	—	Policy end date
`policy_holder_id`	String	Reference Key	→ `policyholders._id`
`branch_id`	String	Reference Key	→ `branches._id`
`event_id`	String	Reference Key	→ `events._id`

Listing 4. *Policy Document Example*

```
{
  "_id": "P201",
  "policy_number": "POL1001",
  "premium_amount": 12000,
  "coverage_amount": 500000,
  "start_date": ISODate("2024-01-01"),
  "end_date": ISODate("2025-01-01"),
  "policy_holder_id": "PH001",
  "branch_id": "B101",
  "event_id": "E101"
```

```
}
```

6.6 Entity 5: ServiceProvider

Description: Third-party organisations handling claims (hospitals, garages, inspection firms).

Collection Name: `service_providers`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique provider identifier
`provider_name`	String	—	Name of service provider
`provider_type`	String	—	Hospital / Garage / Inspection
`address.street`	String	—	Street (sub-document)
`address.city`	String	—	City (sub-document)
`address.state`	String	—	State (sub-document)
`phone`	String	—	Contact number
`license_no`	String	Unique Key	Professional licence number
`recommended_amount`	Decimal	—	Recommended settlement amount
`remarks`	String	—	Notes on provider

Listing 5. *ServiceProvider Document Example*

```
{
  "_id": "SP301",
  "provider_name": "Apollo Hospital",
  "provider_type": "Hospital",
  "address": { "street": "Greams Road", "city": "Chennai",
               "state": "Tamil Nadu", "pincode": "600006" },
  "phone": "04428293333",
  "license_no": "HOSP001",
  "recommended_amount": 115000,
  "remarks": "Verified hospital for medical claims"
}
```

6.7 Entity 6: Claim (with Embedded Documents and Status History)

Description: Central transactional document. Embeds ClaimDocument and ClaimStatusHistory arrays. Carries EventID directly as it is triggered by an event.

Collection Name: `claims`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique claim identifier
`claim_number`	String	Unique Key	Human-readable claim reference
`claim_date`	Date	—	Date claim was filed
`claim_amount`	Decimal	—	Amount claimed
`claim_status`	String	—	Current workflow status
`event_id`	String	Reference Key	→ `events._id` (TRIGGERS)
`policy_id`	String	Reference Key	→ `policies._id`
`provider_id`	String	Reference Key	→ `service_providers._id`
`documents[]`	Array<Doc>	Embedded	Supporting claim documents
`status_history[]`	Array<Doc>	Embedded	Status transition audit trail

Embedded Entity: ClaimDocument (inside `claims.documents[]`)

> **Design Note:** In the ER model, ClaimDocument is a *strong entity* with its own DocumentID as primary key and ClaimID as foreign key. In the NoSQL design, because ClaimDocument is always accessed in the context of its parent claim and has no independent lifecycle, it is stored as an embedded array within the Claim document. DocumentID is retained as a unique identifier within the array.

Attribute	Data Type	Key Type	Description
`document_id`	String	Unique within Claim	Document identifier
`file_path`	String	—	File storage path
`document_type`	String	—	Medical Report / Bill / Photo
`upload_date`	Date	—	Date uploaded

Embedded Entity: ClaimStatusHistory (inside `claims.status_history[]`)

> **Design Note:** In the ER model, ClaimStatusHistory is a *strong entity* with its own StatusID as primary key. In the NoSQL design, it is stored as an embedded append-only array within the Claim document, providing a built-in audit trail without requiring a separate collection lookup.

Attribute	Data Type	Key Type	Description
status_id	String	Unique within Claim	Status entry identifier
status	String	—	Submitted / Under Review / Approved / Rejected
status_date	Date	—	Date of this status transition
updated_by	String	—	System / Surveyor / Manager

Listing 6. *Claim Document with Embedded Arrays*

```
{
  "_id": "C401",
  "claim_number": "CLM5001",
  "claim_date": ISODate("2024-07-05"),
  "claim_amount": 120000,
  "claim_status": "Approved",
  "event_id": "E101",
  "policy_id": "P201",
  "provider_id": "SP301",
  "documents": [
    { "document_id": "D01", "file_path": "/docs/medical_report.pdf",
      "document_type": "Medical Report", "upload_date": ISODate
         ("2024-07-05") },
    { "document_id": "D02", "file_path": "/docs/hospital_bill.pdf",
      "document_type": "Hospital Bill",  "upload_date": ISODate
         ("2024-07-05") }
  ],
  "status_history": [
    { "status_id": "ST801", "status": "Submitted",
      "status_date": ISODate("2024-07-05"), "updated_by": "System"
           },
    { "status_id": "ST802", "status": "Under Review",
      "status_date": ISODate("2024-07-06"), "updated_by": "Surveyor"
          },
    { "status_id": "ST803", "status": "Approved",
      "status_date": ISODate("2024-07-08"), "updated_by": "Manager"
          }
  ]
}
```

6.8 Entity 7: Surveyor

Description: Authorised personnel responsible for claim verification and assessment.

Collection Name: `surveyors`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique surveyor identifier
`name`	String	—	Full name of surveyor
`license_no`	String	Unique Key	Professional licence number
`phone`	String	—	Contact number

Listing 7. *Surveyor Document Example*

```
{
  "_id": "S501",
  "name": "Rajesh Sharma",
  "license_no": "LIC1001",
  "phone": "9001122334"
}
```

6.9 Entity 8: ClaimReview

Description: Review activity by a surveyor on a claim. Resolves the M:N relationship between Claim and Surveyor.

Collection Name: `claim_reviews`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique review identifier
`claim_id`	String	Reference Key	→ `claims._id`
`surveyor_id`	String	Reference Key	→ `surveyors._id`
`review_date`	Date	—	Date of review
`recommended_amount`	Decimal	—	Recommended settlement amount
`remarks`	String	—	Review comments

Listing 8. *ClaimReview Document Example*

```
{
  "_id": "R601",
  "claim_id": "C401",
  "surveyor_id": "S501",
  "review_date": ISODate("2024-07-06"),
  "recommended_amount": 115000,
  "remarks": "Medical expenses verified and validated"
}
```

6.10 Entity 9: Payment

Description: Financial settlement for approved claims. Stored separately for audit and financial traceability.

Collection Name: `payments`

Attribute	Data Type	Key Type	Description
`_id`	String	Primary Key	Unique payment identifier
`claim_id`	String	Reference Key	→ `claims._id`
`payment_date`	Date	—	Date of payment
`payment_amount`	Decimal	—	Amount paid
`payment_mode`	String	—	Bank Transfer / UPI / Cheque
`transaction_ref_no`	String	Unique Key	Bank transaction reference

Listing 9. *Payment Document Example*

```
{
  "_id": "PAY701",
  "claim_id": "C401",
  "payment_date": ISODate("2024-07-10"),
  "payment_amount": 115000,
  "payment_mode": "Bank Transfer",
  "transaction_ref_no": "TXN987654"
}
```

6.11 Summary of Entities and Storage

Entity	Type	Storage Location
PolicyHolder	Primary Collection	`policyholders`
Branch	Primary Collection	`branches`
Event	Primary Collection	`events`
Policy	Primary Collection	`policies`
ServiceProvider	Primary Collection	`service_providers`

Entity	Type	Storage Location
Claim	Primary Collection	`claims`
Surveyor	Primary Collection	`surveyors`
ClaimReview	Primary Collection	`claim_reviews`
Payment	Primary Collection	`payments`
phones[]	Embedded Array	Inside `policyholders`
address{}	Embedded Sub-doc	Inside `policyholders` and `service_providers`
documents[] (ClaimDocument)	Embedded Array	Inside `claims`
status_history[] (ClaimStatusHistory)	Embedded Array	Inside `claims`

6.12 Key Constraints Summary

Key Type	Collection	Attribute / Reference
Primary Key	All 9 collections	`_id` field of each collection
Unique Key	policyholders	`aadhaar_no`
Unique Key	policies	`policy_number`
Unique Key	claims	`claim_number`
Unique Key	surveyors	`license_no`
Unique Key	service_providers	`license_no`
Unique Key	payments	`transaction_ref_no`
Reference Key	policies	`policy_holder_id` → `policyholders._id`
Reference Key	policies	`branch_id` → `branches._id`
Reference Key	policies	`event_id` → `events._id`
Reference Key	claims	`event_id` → `events._id` (TRIGGERS)

Key Type	Collection	Attribute / Reference
Reference Key	claims	`policy_id` → `policies._id`
Reference Key	claims	`provider_id` → `service_providers._id`
Reference Key	claim_reviews	`claim_id` → `claims._id`
Reference Key	claim_reviews	`surveyor_id` → `surveyors._id`
Reference Key	payments	`claim_id` → `claims._id`

7. Relationships with Types, Implementation, and Participation

7.1 PolicyHolder – Policy (HAS)

Type: 1:N **Implementation:** Referenced (`policy_holder_id` in policies)
Participation: PolicyHolder — Partial; Policy — Total.
One policyholder owns multiple policies. Each policy must belong to exactly one policyholder.

7.2 Branch – Policy (MANAGES)

Type: 1:N **Implementation:** Referenced (`branch_id` in policies)
Participation: Branch — Partial; Policy — Total.
One branch manages multiple policies. Each policy is issued by exactly one branch.

7.3 Event – Policy (CATEGORIZES)

Type: 1:N **Implementation:** Referenced (`event_id` in policies)
Participation: Event — Partial; Policy — Total.
One event type categorises multiple policies. This is the core mechanism that eliminates subclass tables.

7.4 Policy – Claim (FILES)

Type: 1:N **Implementation:** Referenced (`policy_id` in claims)
Participation: Policy — Partial; Claim — Total.
One policy can have multiple claims filed against it. Each claim must reference a valid policy.

7.5 Claim – ServiceProvider (HANDLES)

Type: N:1 **Implementation:** Referenced (`provider_id` in claims)
Participation: Claim — Total; ServiceProvider — Partial.
Each claim is handled by one service provider. A provider may handle claims from multiple policyholders.

7.6 Event – Claim (TRIGGERS)

Type: 1:N **Implementation:** Referenced (`event_id` in claims)
Participation: Event — Partial; Claim — Total.
Each claim is triggered by exactly one event type. This allows the claim to carry its insurance category directly without joining through Policy.

7.7 Claim – ClaimDocument (HAS_DOCUMENT)

Type: 1:N **Implementation:** Embedded (`documents[]` array in claims)
Participation: Claim — Partial; ClaimDocument — Total.
One claim has multiple supporting documents. Embedded because documents are always accessed with their claim and are bounded in size.

7.8 Claim – ClaimStatusHistory (HAS_HISTORY)

Type: 1:N **Implementation:** Embedded (`status_history[]` array in claims)
Participation: Both — Total.
One claim has multiple status transitions forming an append-only audit trail. Embedded for atomic status updates.

7.9 Claim – ClaimReview (REVIEWED_BY)

Type: 1:N **Implementation:** Referenced (`claim_id` in claim_reviews)
Participation: Claim — Partial; ClaimReview — Total.
One claim can be reviewed multiple times by different surveyors.

7.10 Surveyor – ClaimReview (PERFORMS)

Type: 1:N **Implementation:** Referenced (`surveyor_id` in claim_reviews)
Participation: Surveyor — Partial; ClaimReview — Total.
One surveyor performs reviews on multiple claims.

7.11 Claim – Payment (SETTLED_BY)

Type: 1:1 **Implementation:** Referenced (`claim_id` in payments)
Participation: Claim — Partial; Payment — Total.
An approved claim is settled by exactly one payment. Not all claims result in payment.

7.12 Final Relationship Summary Table

Relationship	Entities	Type	Implementati	Participation
HAS	PolicyHolder – Policy	1:N	Referenced	PH Partial; Policy Total
MANAGES	Branch – Policy	1:N	Referenced	Branch Partial; Policy Total
CATEGORIZES	Event – Policy	1:N	Referenced	Event Partial; Policy Total
FILES	Policy – Claim	1:N	Referenced	Policy Partial; Claim Total
HANDLES	Claim – ServiceProvider	N:1	Referenced	Claim Total; SP Partial
TRIGGERS	Event – Claim	1:N	Referenced	Event Partial; Claim Total
HAS_DOCUMENT	Claim – ClaimDocument	1:N	Embedded Array	Claim Partial; Doc Total
HAS_HISTORY	Claim – StatusHistory	1:N	Embedded Array	Both Total
REVIEWED_BY	Claim – ClaimReview	1:N	Referenced	Claim Partial; Review Total
PERFORMS	Surveyor – ClaimReview	1:N	Referenced	Surveyor Partial; Review Total
SETTLED_BY	Claim – Payment	1:1	Referenced	Claim Partial; Payment Total

8. Assignment 1

8.1 (a) Identify Entities, Attributes, and Relationships

Identified Collections (Entities) and Key Attributes

Collection	Key Attributes
policyholders	_id, name, dob, gender, aadhaar_no, phones[], address{}
branches	_id, branch_name, location, manager_name, contact
events	_id, event_name, event_description, risk_level

Collection	Key Attributes
policies	_id, policy_number, premium_amount, coverage_amount, start_date, end_date, policy_holder_id, branch_id, event_id
service_providers	_id, provider_name, provider_type, address{}, phone, license_no, recommended_amount, remarks
claims	_id, claim_number, claim_date, claim_amount, claim_status, event_id, policy_id, provider_id, documents[], status_history[]
surveyors	_id, name, license_no, phone
claim_reviews	_id, claim_id, surveyor_id, review_date, recommended_amount, remarks
payments	_id, claim_id, payment_date, payment_amount, payment_mode, transaction_ref_no

Relationships with Cardinality and Participation

Relationship	Entities	Cardinal	Participation
HAS	PolicyHolder – Policy	1:N	PolicyHolder Partial, Policy Total
MANAGES	Branch – Policy	1:N	Branch Partial, Policy Total
CATEGORIZES	Event – Policy	1:N	Event Partial, Policy Total
FILES	Policy – Claim	1:N	Policy Partial, Claim Total
HANDLES	Claim – ServiceProvider	N:1	Claim Total, SP Partial
TRIGGERS	Event – Claim	1:N	Event Partial, Claim Total
HAS_DOCUMENT	Claim – ClaimDocument	1:N	Claim Partial, Document Total
HAS_HISTORY	Claim – StatusHistory	1:N	Both Total
REVIEWED_BY	Claim – ClaimReview	1:N	Claim Partial, Review Total

Relationship	Entities	Cardinal	Participation
PERFORMS	Surveyor – ClaimReview	1:N	Surveyor Partial, Review Total
SETTLED_BY	Claim – Payment	1:1	Claim Partial, Payment Total

Data Grouping and Embedding

Embedded inside `policyholders`:

- `phones[]` — multivalued attribute
- `address`{} — composite attribute

Embedded inside `claims`:

- `documents[]` — ClaimDocument (strong entity in ER model; embedded in NoSQL for performance)
- `status_history[]` — ClaimStatusHistory (strong entity in ER model; embedded in NoSQL for audit trail)

Referenced (separate collections):

- All other entities are maintained as independent collections referenced by ID

8.2 (b) Conceptual Diagram (NoSQL High-Level Model)

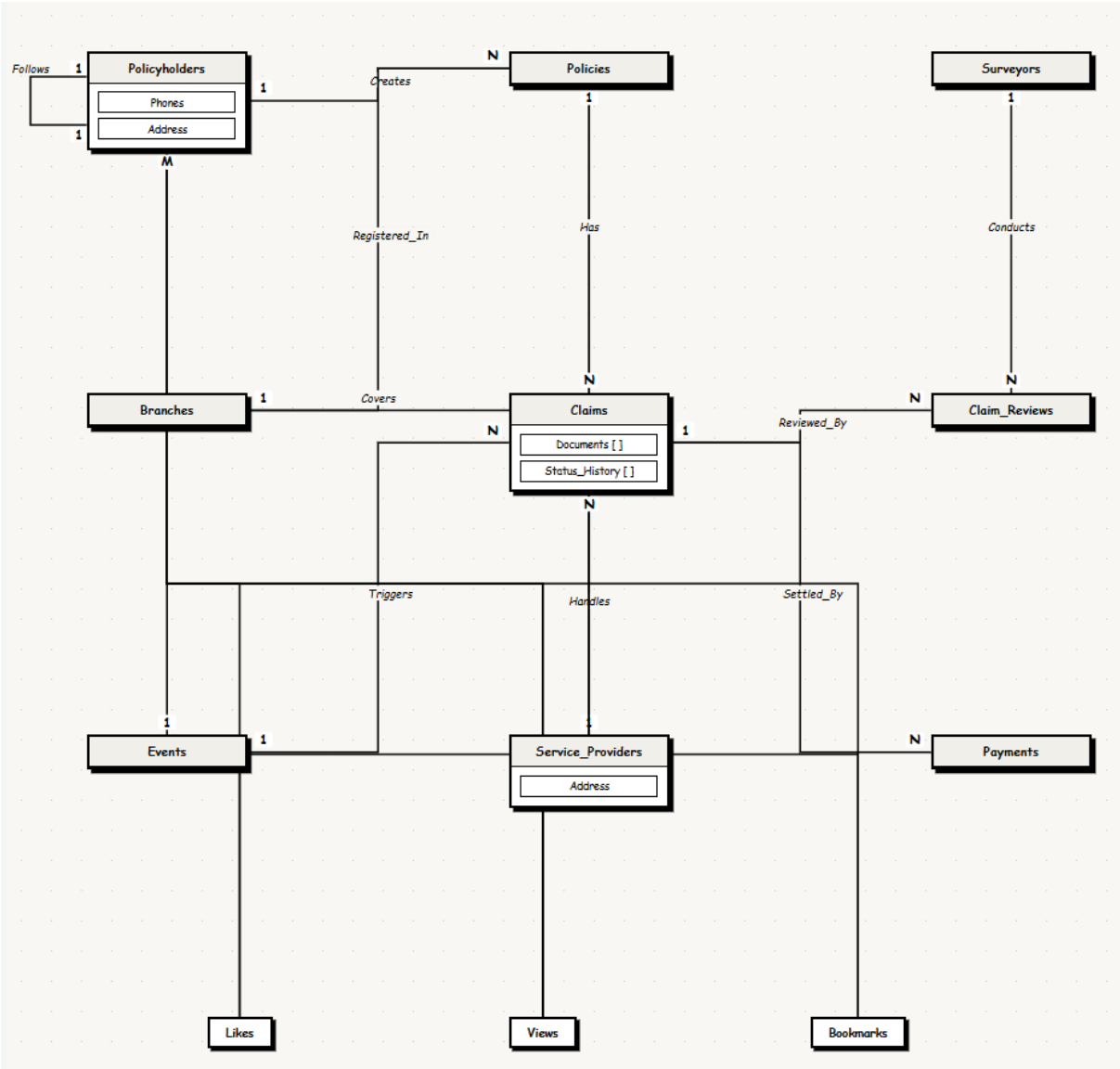

Figure 2. *Conceptual NoSQL High-Level Model for ICPS*

Logical and Structural MongoDB Model

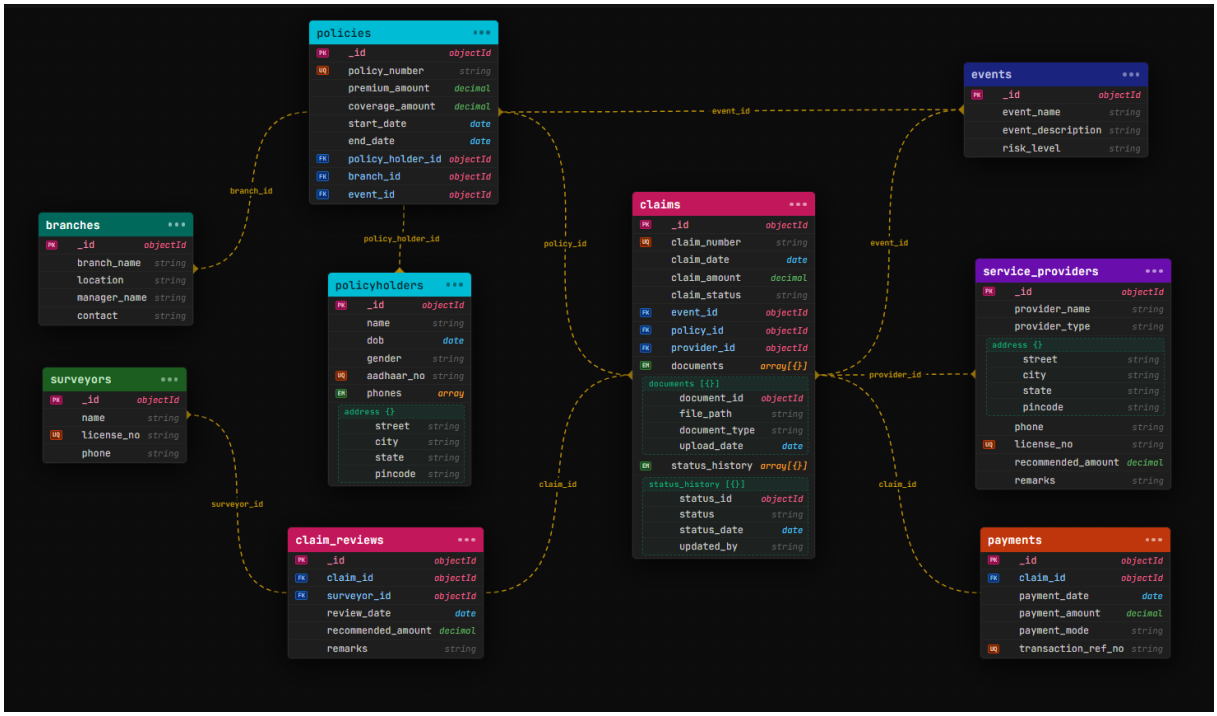

Figure 3. *Logical and Structural MongoDB Model for ICPS*

Collections Structure Summary

Collection	Strategy	Justification
policyholders	Reference	Shared across many policies; independent lifecycle
branches	Reference	Shared across many policies
events	Reference	Classification table; reused across policies and claims
policies	Reference	References PH, Branch, Event
service_providers	Reference	Shared across multiple claims
claims	Reference + Embed	References Policy, SP, Event; embeds documents[] and status_history[]
surveyors	Reference	Independent entity; shared across reviews
claim_reviews	Reference	References Claim and Surveyor
payments	Reference	Independent financial record; references Claim

8.3 (c) Map to Physical Schema

PolicyHolders Collection Schema

Listing 10. *Physical Schema: policyholders*

```
{
  "_id": String,           // Primary Key
  "name": String,          // Required
  "dob": Date,             // Required
  "gender": String,        // Required: M / F / Other
  "aadhaar_no": String,    // Unique, Required
  "phones": [String],      // Embedded array
  "address": { "street": String, "city": String,
               "state": String, "pincode": String }
}
```

Branches Collection Schema

Listing 11. *Physical Schema: branches*

```
{
  "_id": String, "branch_name": String, "location": String,
  "manager_name": String, "contact": String
}
```

Events Collection Schema

Listing 12. *Physical Schema: events*

```
{
  "_id": String, "event_name": String,
  "event_description": String,
  "risk_level": String   // Medium / High / Very High
}
```

Policies Collection Schema

Listing 13. *Physical Schema: policies*

```
{
  "_id": String,
  "policy_number": String,      // Unique
  "premium_amount": Decimal,
  "coverage_amount": Decimal,
  "start_date": Date, "end_date": Date,
  "policy_holder_id": String,   // FK -> policyholders._id
  "branch_id": String,          // FK -> branches._id
  "event_id": String            // FK -> events._id
}
```

Service Providers Collection Schema

Listing 14. *Physical Schema: service_providers*

```
{
  "_id": String,
  "provider_name": String, "provider_type": String,
  "address": { "street": String, "city": String,
               "state": String, "pincode": String },
  "phone": String,
  "license_no": String,          // Unique
  "recommended_amount": Decimal,
  "remarks": String
}
```

Claims Collection Schema

Listing 15. *Physical Schema: claims*

```
{
  "_id": String,
  "claim_number": String,        // Unique
  "claim_date": Date,
  "claim_amount": Decimal,
  "claim_status": String,        // Submitted/Under Review/Approved/
     Rejected
  "event_id": String,            // FK -> events._id  (TRIGGERS
     relationship)
  "policy_id": String,           // FK -> policies._id
  "provider_id": String,         // FK -> service_providers._id
  "documents": [
    { "document_id": String, "file_path": String,
      "document_type": String, "upload_date": Date }
  ],
  "status_history": [
    { "status_id": String, "status": String,
      "status_date": Date, "updated_by": String }
  ]
}
```

Surveyors Collection Schema

Listing 16. *Physical Schema: surveyors*

```
{
  "_id": String, "name": String,
  "license_no": String,   // Unique
  "phone": String
```

```
}
```

ClaimReviews Collection Schema

Listing 17. *Physical Schema: claim_reviews*

```
{
  "_id": String,
  "claim_id": String,              // FK -> claims._id
  "surveyor_id": String,           // FK -> surveyors._id
  "review_date": Date,
  "recommended_amount": Decimal,
  "remarks": String
}
```

Payments Collection Schema

Listing 18. *Physical Schema: payments*

```
{
  "_id": String,
  "claim_id": String,             // FK -> claims._id
  "payment_date": Date,
  "payment_amount": Decimal,
  "payment_mode": String,
  "transaction_ref_no": String    // Unique
}
```

Partition Key and Indexes

- Partition Key: `claims._id`
- Indexes: `aadhaar_no` (unique), `policy_number` (unique), `claim_number` (unique), `license_no` ×2 (unique), `transaction_ref_no` (unique)
- Non-unique indexes: `policy_holder_id`, `branch_id`, `event_id` (policies), `event_id` (claims), `policy_id`, `claim_status`, `provider_id`, `claim_id` ×2, `surveyor_id`

8.4 (d) Functional Dependencies, Query Patterns, and Key Identification

Primary Keys

- All 9 collections: `_id` is the primary key

Unique Keys

- `policyholders.aadhaar_no`, `policies.policy_number`, `claims.claim_number`
- `surveyors.license_no`, `service_providers.license_no`, `payments.transaction_ref_no`

Functional Dependencies

policy_holder_id →	name, dob, gender, aadhaar_no, phones, address
aadhaar_no →	policy_holder_id
branch_id →	branch_name, location, manager_name, contact
event_id →	event_name, event_description, risk_level
policy_id →	policy_number, premium_amount, coverage_amount, start_date, end_date, policy_holder_id, branch_id, event_id
policy_number →	policy_id
provider_id →	provider_name, provider_type, address, phone, license_no, recommended_amount, remarks
license_no (SP) →	provider_id
claim_id →	claim_number, claim_date, claim_amount, claim_status, event_id, policy_id, provider_id, documents[], status_history[]
claim_number →	claim_id
surveyor_id →	name, license_no, phone
license_no (Sur) →	surveyor_id
review_id →	claim_id, surveyor_id, review_date, recommended_amount, remarks
payment_id →	claim_id, payment_date, payment_amount, payment_mode, transaction_ref_no
transaction_ref_no →	payment_id

Query Patterns

Listing 19. *Retrieve complete claim with embedded docs and history*

```
db.claims.findOne({ _id: "C401" })
```

Listing 20. *All claims for a policy*

```
db.claims.find({ policy_id: "P201" })
```

Listing 21. *All policies by a policyholder*

```
db.policies.find({ policy_holder_id: "PH001" })
```

Listing 22. *All Medical policies by event type*

```
db.policies.find({ event_id: "E101" })
```

Listing 23. *All claims triggered by a specific event*

```
db.claims.find({ event_id: "E101" })
```

Listing 24. *All policies by branch*

```
db.policies.find({ branch_id: "B101" })
```

Listing 25. *All pending claims*

```
db.claims.find({ claim_status: "Submitted" })
```

Listing 26. *All reviews for a claim*

```
db.claim_reviews.find({ claim_id: "C401" })
```

Listing 27. *Payment for a claim*

```
db.payments.findOne({ claim_id: "C401" })
```

Embedding vs Referencing Justification

Embedded:

- `documents[]` and `status_history[]` in claims — always accessed with the claim; bounded in size; no independent access pattern; enables single-query retrieval.
- `phones[]` in policyholders — multivalued; always with policyholder; eliminates relational table.
- `address{}` in policyholders and service_providers — composite; not shared; eliminates `Location` table.

Referenced:

- `policy_holder_id`, `branch_id`, `event_id` in policies — shared entities; avoid duplication.
- `event_id`, `policy_id`, `provider_id` in claims — independent lifecycles; shared entities.
- `surveyor_id` in claim_reviews — surveyor exists independently; reviews many claims.
- `claim_id` in payments — independent financial record.

Denormalisation Trade-off

`claim_status` is stored both as a direct field (fast lookup) and as the latest entry in `status_history[]` (audit trail). Updates must maintain both atomically using a single `updateOne()` with `$set` and `$push`. Acceptable in NoSQL for read performance gains.

Final NoSQL Architecture Summary

Collection	Embedded Entities	References
policyholders	phones[], address{}	—
branches	—	—
events	—	—
policies	—	policy_holder_id, branch_id, event_id
service_providers	address{}	—
claims	documents[], status_history[]	event_id, policy_id, provider_id
surveyors	—	—
claim_reviews	—	claim_id, surveyor_id
payments	—	claim_id

9. Functional Dependencies and Key Identification

9.1 Introduction

A functional dependency $X \to Y$ states that values of X uniquely determine values of Y. FDs help identify candidate keys, eliminate redundancy, and ensure data consistency. In the NoSQL model, embedded arrays derive their identity from the parent document's primary key and carry no independent FDs.

9.2 Functional Dependencies per Collection

PolicyHolder

$$\texttt{policy_holder_id} \to \texttt{name, dob, gender, aadhaar_no, phones, address}$$
$$\texttt{aadhaar_no} \to \texttt{policy_holder_id}$$

Branch

$$\texttt{branch_id} \to \texttt{branch_name, location, manager_name, contact}$$

Event

$$\texttt{event_id} \rightarrow \texttt{event_name},\ \texttt{event_description},\ \texttt{risk_level}$$

Policy

$$\begin{aligned}\texttt{policy_id} &\rightarrow \texttt{policy_number},\ \texttt{premium_amount},\ \texttt{coverage_amount},\\ &\quad \texttt{start_date},\ \texttt{end_date},\ \texttt{policy_holder_id},\\ &\quad \texttt{branch_id},\ \texttt{event_id}\\ \texttt{policy_number} &\rightarrow \texttt{policy_id}\end{aligned}$$

ServiceProvider

$$\begin{aligned}\texttt{provider_id} &\rightarrow \texttt{provider_name},\ \texttt{provider_type},\ \texttt{address},\\ &\quad \texttt{phone},\ \texttt{license_no},\ \texttt{recommended_amount},\ \texttt{remarks}\\ \texttt{license_no (SP)} &\rightarrow \texttt{provider_id}\end{aligned}$$

Claim

$$\begin{aligned}\texttt{claim_id} &\rightarrow \texttt{claim_number},\ \texttt{claim_date},\ \texttt{claim_amount},\\ &\quad \texttt{claim_status},\ \texttt{event_id},\ \texttt{policy_id},\ \texttt{provider_id}\\ \texttt{claim_number} &\rightarrow \texttt{claim_id}\end{aligned}$$

Surveyor

$$\begin{aligned}\texttt{surveyor_id} &\rightarrow \texttt{name},\ \texttt{license_no},\ \texttt{phone}\\ \texttt{license_no (Sur)} &\rightarrow \texttt{surveyor_id}\end{aligned}$$

ClaimReview

$$\begin{aligned}\texttt{review_id} &\rightarrow \texttt{claim_id},\ \texttt{surveyor_id},\ \texttt{review_date},\\ &\quad \texttt{recommended_amount},\ \texttt{remarks}\end{aligned}$$

Payment

$$\begin{aligned}\texttt{payment_id} &\rightarrow \texttt{claim_id},\ \texttt{payment_date},\ \texttt{payment_amount},\\ &\quad \texttt{payment_mode},\ \texttt{transaction_ref_no}\\ \texttt{transaction_ref_no} &\rightarrow \texttt{payment_id}\end{aligned}$$

9.3 Derivation of Minimal Cover

Step 1: Decompose RHS to single attributes. Already done above.

Step 2: Remove extraneous LHS attributes. All LHS are single attributes (primary keys); none are extraneous.

Step 3: Remove redundant dependencies.

- `aadhaar_no` → `policy_holder_id` and `policy_holder_id` → X makes direct `aadhaar_no` → X redundant.
- `policy_number` → `policy_id` and `policy_id` → X makes `policy_number` → X (for non-key X) redundant.
- Similarly for `claim_number`, `license_no` (both), and `transaction_ref_no`.

Final Minimal Cover

`policy_holder_id` →	`name`, `dob`, `gender`, `aadhaar_no`, `phones`, `address`
`aadhaar_no` →	`policy_holder_id`
`branch_id` →	`branch_name`, `location`, `manager_name`, `contact`
`event_id` →	`event_name`, `event_description`, `risk_level`
`policy_id` →	`policy_number`, `premium_amount`, `coverage_amount`, `start_date`, `end_date`, `policy_holder_id`, `branch_id`, `event_id`
`policy_number` →	`policy_id`
`provider_id` →	`provider_name`, `provider_type`, `address`, `phone`, `license_no`, `recommended_amount`, `remarks`
`license_no (SP)` →	`provider_id`
`claim_id` →	`claim_number`, `claim_date`, `claim_amount`, `claim_status`, `event_id`, `policy_id`, `provider_id`
`claim_number` →	`claim_id`
`surveyor_id` →	`name`, `license_no`, `phone`
`license_no (Sur)` →	`surveyor_id`
`review_id` →	`claim_id`, `surveyor_id`, `review_date`, `recommended_amount`, `remarks`
`payment_id` →	`claim_id`, `payment_date`, `payment_amount`, `payment_mode`, `transaction_ref_no`
`transaction_ref_no` →	`payment_id`

9.4 Key Identification

Collection	Candidate Keys	Primary Key Chosen
policyholders	`policy_holder_id`, `aadhaar_no`	`policy_holder_id`
branches	`branch_id`	`branch_id`
events	`event_id`	`event_id`
policies	`policy_id`, `policy_number`	`policy_id`
service_providers	`provider_id`, `license_no`	`provider_id`
claims	`claim_id`, `claim_number`	`claim_id`
surveyors	`surveyor_id`, `license_no`	`surveyor_id`
claim_reviews	`review_id`	`review_id`
payments	`payment_id`, `transaction_ref_no`	`payment_id`

10. Assignment 2: Optimised NoSQL Database Schema Design

10.1 Objective of Schema Optimisation

The objective is to optimise the ICPS database for high read performance, efficient query execution, horizontal scalability, and reduced operations per claim lookup while maintaining data consistency.

10.2 Frequently Executed Query Patterns

Query	Frequency	Optimisation Strategy
Retrieve claim with documents and history	Very High	Embed `documents[]` and `status_history[]`
All claims for a policy	Very High	Index: `claims.policy_id`
All policies by policyholder	Very High	Index: `policies.policy_holder_id`
All claims by event type	High	Index: `claims.event_id`
Policies by event type	High	Index: `policies.event_id`
Policies by branch	High	Index: `policies.branch_id`
Pending / under-review claims	High	Index: `claims.claim_status`

Query	Frequency	Optimisation Strategy
Payment for a claim	Moderate	Index: `payments.claim_id`
Reviews for a claim	Moderate	Index: `claim_reviews.claim_id`

10.3 Optimised Claims Collection Schema

Listing 28. *Optimised claims Collection Schema*

```
{
  "_id": String,
  "claim_number": String,     // Unique index
  "claim_date": Date,
  "claim_amount": Decimal,
  "claim_status": String,     // Indexed
  "event_id": String,         // Indexed - TRIGGERS relationship
  "policy_id": String,        // Indexed
  "provider_id": String,      // Indexed
  "documents": [              // Embedded - strong entity
      ClaimDocument
    { "document_id": String, "file_path": String,
      "document_type": String, "upload_date": Date }
  ],
  "status_history": [         // Embedded - strong entity
      ClaimStatusHistory
    { "status_id": String, "status": String,
      "status_date": Date, "updated_by": String }
  ]
}
```

10.4 Embedding vs Referencing Optimisation Decisions

Embedded Data

Embedded Entity	Reason
`documents[]` in claims	Always with claim; no independent access; bounded size
`status_history[]` in claims	Append-only audit trail; always with claim
`phones[]` in policyholders	Multivalued; always with policyholder
`address{}` in policyholders	Composite; not shared

Embedded Entity	Reason
address{} in service_providers	Not shared across documents

Referenced Data

Reference	Reason
policy_holder_id in policies	Shared; avoid duplication
branch_id in policies	One branch manages many policies
event_id in policies	Universal classifier; shared
event_id in claims	TRIGGERS relationship; event independent of claim
policy_id in claims	One policy generates many claims
provider_id in claims	Independent entity; shared across claims
surveyor_id in claim_reviews	Independent; shared across reviews
claim_id in payments	Independent financial record

10.5 Denormalisation Strategy

`claim_status` is stored as a direct field (fast current status lookup) and as the latest entry in `status_history[]` (full history). Updated atomically with a single `updateOne()` using `$set` and `$push`.

10.6 Index Optimisation

Listing 29. *Index Creation Commands for All ICPS Collections*

```
// PolicyHolders
db.policyholders.createIndex({ aadhaar_no: 1 }, { unique: true })

// Policies
db.policies.createIndex({ policy_number: 1 }, { unique: true })
db.policies.createIndex({ policy_holder_id: 1 })
db.policies.createIndex({ branch_id: 1 })
db.policies.createIndex({ event_id: 1 })

// Claims
db.claims.createIndex({ claim_number: 1 }, { unique: true })
db.claims.createIndex({ event_id: 1 })
db.claims.createIndex({ policy_id: 1 })
db.claims.createIndex({ claim_status: 1 })
db.claims.createIndex({ provider_id: 1 })
```

```

// ClaimReviews
db.claim_reviews.createIndex({ claim_id: 1 })
db.claim_reviews.createIndex({ surveyor_id: 1 })

// Payments
db.payments.createIndex({ transaction_ref_no: 1 }, { unique: true })
db.payments.createIndex({ claim_id: 1 })

// Surveyors
db.surveyors.createIndex({ license_no: 1 }, { unique: true })

// ServiceProviders
db.service_providers.createIndex({ license_no: 1 }, { unique: true
    })
```

10.7 Partition Key and Scalability

Partition Key:	`claims._id`
Justification:	Uniform distribution; each claim is self-contained with embedded sub-entities
Shard Command:	`sh.shardCollection("icps.claims", { _id: 1 })`

10.8 Performance Optimisation Benefits

- **Single-query retrieval:** Complete claim with all documents and history in one `findOne()`.
- **Reduced latency:** Strategic indexes minimise collection scans.
- **Horizontal scalability:** Sharding on `claim_id` distributes load uniformly.
- **Efficient writes:** `$push` for appending documents and status entries is O(1).
- **Flexible expansion:** New insurance types require one new Event document only.

10.9 Data Consistency Considerations

- MongoDB document-level atomicity ensures `claim_status` update and `status_history[]` push happen atomically.
- Reference IDs enforce referential integrity at application level.
- Unique indexes enforce business-critical field uniqueness.

10.10 Final Optimised Schema Architecture Summary

Collection	Embedded Entities	References
policyholders	phones[], address{}	—
branches	—	—
events	—	—
policies	—	policy_holder_id, branch_id, event_id
service_providers	address{}	—
claims	documents[], status_history[]	event_id, policy_id, provider_id
surveyors	—	—
claim_reviews	—	claim_id, surveyor_id
payments	—	claim_id

11. Implementation Details

11.1 Collection Initialisation

Listing 30. *Creating All ICPS Collections*

```
db.createCollection("policyholders")
db.createCollection("branches")
db.createCollection("events")
db.createCollection("policies")
db.createCollection("service_providers")
db.createCollection("claims")
db.createCollection("surveyors")
db.createCollection("claim_reviews")
db.createCollection("payments")
```

11.2 Sample CRUD Operations

Create Operations

Listing 31. *Register a PolicyHolder*

```
db.policyholders.insertOne({
  _id: "PH001", name: "Arjun Kumar",
  dob: new Date("1990-05-12"), gender: "M",
  aadhaar_no: "456789123456",
  phones: ["9876543210"],
  address: { street: "Anna Nagar", city: "Chennai",
             state: "Tamil Nadu", pincode: "600040" }
})
```

Listing 32. *Create a Policy*

```
db.policies.insertOne({
  _id: "P201", policy_number: "POL1001",
  premium_amount: 12000, coverage_amount: 500000,
  start_date: new Date("2024-01-01"),
  end_date: new Date("2025-01-01"),
  policy_holder_id: "PH001",
  branch_id: "B101",
  event_id: "E101"
})
```

Listing 33. *Submit a Claim with Initial Status History*

```
db.claims.insertOne({
  _id: "C401", claim_number: "CLM5001",
  claim_date: new Date("2024-07-05"),
  claim_amount: 120000, claim_status: "Submitted",
  event_id: "E101",
  policy_id: "P201",
  provider_id: "SP301",
  documents: [],
  status_history: [{
    status_id: "ST801", status: "Submitted",
    status_date: new Date("2024-07-05"), updated_by: "System"
  }]
})
```

Listing 34. *Add a Document to a Claim*

```
db.claims.updateOne(
  { _id: "C401" },
  { $push: { documents: {
    document_id: "D01",
    file_path: "/docs/medical_report.pdf",
    document_type: "Medical Report",
    upload_date: new Date("2024-07-05")
  }}}
)
```

Listing 35. *Record a Claim Review*

```
db.claim_reviews.insertOne({
  _id: "R601", claim_id: "C401", surveyor_id: "S501",
  review_date: new Date("2024-07-06"),
  recommended_amount: 115000,
  remarks: "Medical expenses verified"
})
```

Listing 36. *Process a Payment*

```
db.payments.insertOne({
  _id: "PAY701", claim_id: "C401",
  payment_date: new Date("2024-07-10"),
  payment_amount: 115000, payment_mode: "Bank Transfer",
  transaction_ref_no: "TXN987654"
})
```

Read Operations

Listing 37. *Retrieve Complete Claim*

```
db.claims.findOne({ _id: "C401" })
```

Listing 38. *All Claims for a Policy*

```
db.claims.find({ policy_id: "P201" })
```

Listing 39. *All Claims Triggered by an Event*

```
db.claims.find({ event_id: "E101" })
```

Listing 40. *All Pending Claims*

```
db.claims.find({ claim_status: "Submitted" })
```

Update Operations

Listing 41. *Update Claim Status and Append to History (Atomic)*

```
db.claims.updateOne(
  { _id: "C401" },
  {
    $set:  { claim_status: "Approved" },
    $push: { status_history: {
      status_id: "ST803", status: "Approved",
      status_date: new Date("2024-07-08"), updated_by: "Manager"
    }}
  }
)
```

Listing 42. *Add a New Phone Number*

```
db.policyholders.updateOne(
  { _id: "PH001" },
  { $push: { phones: "9001122334" } }
)
```

Delete Operations

Listing 43. *Remove a Document from a Claim*

```
db.claims.updateOne(
  { _id: "C401" },
  { $pull: { documents: { document_id: "D01" } } }
)
```

Listing 44. *Delete a Rejected Claim*

```
db.claims.deleteOne({ _id: "C401", claim_status: "Rejected" })
```

12. Conclusion

The redesigned NoSQL document-based schema for the Insurance Claim Processing System (ICPS) successfully delivers the following outcomes:

- **All Strong Entities:** The updated ER design models all entities — including ClaimDocument and ClaimStatusHistory — as strong entities with independent primary keys. In the NoSQL implementation, these are embedded for performance while retaining their identifiers within the array.
- **TRIGGERS Relationship:** The Event entity participates in both CATEGORIZES (Policy) and TRIGGERS (Claim), allowing the insurance product type to be accessed directly from the Claim document via `event_id` without joining through Policy.
- **Scalability:** Horizontal sharding via `claims._id` handles high claim volumes.
- **Flexibility:** The Event-based classification supports all six insurance product types without schema migration.
- **Performance:** Embedding ClaimDocuments and StatusHistory enables single-query claim lifecycle retrieval.
- **Data Integrity:** Primary keys, unique indexes, and reference IDs maintain consistency across all nine collections.
- **Extensibility:** New products, branches, and providers are added as new documents only.

13. Team Member Contributions

Team Member 1: M. Sujith (3122245001172)

Tasks Completed:

- Designed the complete ER diagram with all strong entities including updated ClaimDocument, ClaimStatusHistory, Branch, Event, and ServiceProvider.

- Established TRIGGERS relationship between Event and Claim.
- Defined embedding strategy for `documents[]` and `status_history[]` within the Claim collection.
- Documented all entity-relationship cardinalities and participation constraints for all 11 relationships.
- Created the conceptual NoSQL high-level diagram.

Team Member 2: S. Sree Krishna (3122245001162)

Tasks Completed:

- Designed MongoDB physical document schemas for all 9 collections with example documents.
- Added `event_id` to Claim collection to reflect TRIGGERS relationship.
- Documented embedding vs referencing decisions with full justification.
- Designed optimised schema architecture including index strategy and partition key.
- Prepared the logical and structural MongoDB model.
- Contributed to CRUD operations section and verified referential consistency.

Team Member 3: Sankar Shailesh (3122245001138)

Tasks Completed:

- Updated functional dependencies per collection to reflect new entities and the TRIGGERS relationship.
- Performed minimal cover derivation with removal of redundant dependencies.
- Derived candidate keys and primary keys for all 9 collections.
- Documented denormalisation strategy and trade-offs.
- Contributed to system specifications, constraints, and final report formatting.